Week 4 Lecture Notes: Autoregressive Models

Hong Qin, Old Dominion University

Fall 2026

Contents

1 Topic 1: Sequential Factorization of Generative Distributions **2**
1.1 Plain-English summary . . . 2
1.2 The chain rule, applied seriously . . . 2
1.3 Training by teacher forcing . . . 2
1.4 Choosing an ordering . . . 3
1.5 Exact likelihood: a real benefit . . . 3
1.6 Payoff . . . 3

2 Topic 2: Recurrent Models and LSTM Intuition **5**
2.1 Plain-English summary . . . 5
2.2 Recurrent Neural Network (RNN) . . . 5
2.3 Gated RNN (GRU) . . . 7
2.4 Long Short-Term Memory (LSTM) . . . 7
2.5 Stacked LSTM . . . 10
2.6 Parameter Sharing and Backpropagation in LSTMs . . . 11
2.7 Comparison and Discussion . . . 12
2.8 Payoff (Key Takeaways) . . . 12

3 Topic 3: Sampling, Conditioning, and Sequence Quality **13**
3.1 Plain-English summary . . . 13
3.2 Decoding strategies: how to pick the next token . . . 13
3.3 Conditioning: steering what the model produces . . . 14
3.4 Quality metrics for autoregressive models . . . 14
3.5 The unavoidable cost: $O(N)$ sampling . . . 15
3.6 Payoff . . . 15

4 Topic 4: Strengths and Limitations of Autoregressive Generation **16**
4.1 Plain-English summary . . . 16
4.2 Where autoregressive shines . . . 16
4.3 Where autoregressive struggles . . . 16
4.4 VAE versus GAN versus autoregressive . . . 17
4.5 Forward reference: transformers and diffusion . . . 17
4.6 Payoff . . . 18

1 Topic 1: Sequential Factorization of Generative Distributions

1.1 Plain-English summary

Week 2 (VAE) chose to introduce a hidden latent variable and train a likelihood lower bound. Week 3 (GAN) chose to skip the likelihood entirely and train an adversarial game. This week introduces the third major family, which makes neither of those choices. Autoregressive models keep the likelihood, drop the latent variable, and instead model a complex high-dimensional distribution as a chain of one-dimensional next-token predictions. This is the family that runs GPT-style language models, WaveNet audio, PixelCNN images, and most modern code-generation systems. The math is simple but the consequences are deep.

1.2 The chain rule, applied seriously

For any joint distribution over a vector $x = (x_1, x_2, \ldots, x_n)$, the chain rule of probability gives an exact factorization:

$$p(x) = p(x_1)\, p(x_2 \mid x_1)\, p(x_3 \mid x_1, x_2) \cdots p(x_n \mid x_1, \ldots, x_{n-1}) = \prod_{i=1}^{n} p(x_i \mid x_{<i})\,, \tag{1}$$

where $x_{<i}$ denotes the prefix $(x_1, \ldots, x_{i-1})$. Equation (1) is always true; it is nothing more than the definition of conditional probability applied $n-1$ times. The autoregressive trick is to take this identity seriously as a modeling strategy: parameterize each conditional $p_\theta(x_i \mid x_{<i})$ with a neural network and train the whole thing by maximum likelihood. The log-likelihood of an observation decomposes into a sum of per-position terms,

$$\log p_\theta(x) = \sum_{i=1}^{n} \log p_\theta(x_i \mid x_{<i})\,, \tag{2}$$

which is exact, computable, and differentiable. There is no ELBO gap (VAE) and no adversarial training instability (GAN). The training objective is simply "maximize the sum of next-token log-probabilities," which is the friendliest objective in deep learning.

1.3 Training by teacher forcing

Equation (2) tells us what to maximize; *teacher forcing* tells us how to evaluate it efficiently. During training, every conditional in the product is evaluated with the *true* prefix from the data, not with the model's own earlier predictions. Concretely, for a training sequence $x = (x_1, \ldots, x_n)$ we feed the ground-truth tokens $x_1, \ldots, x_{i-1}$ into the network, ask it for a distribution over position i, and score that distribution against the true x_i. All n positions can be scored in one forward pass because none of them wait on a sampled output.

For categorical outputs (text tokens, discrete pixel intensities), the network emits a softmax over K classes at each position and the per-sequence loss is the cross-entropy summed over positions,

$$\mathcal{L} = -\sum_{t=1}^{T} \sum_{k=1}^{K} \hat{y}_{t,k} \log y_{t,k}, \tag{3}$$

where $\hat{y}_t$ is the one-hot target at step t and y_t is the predicted distribution. For continuous outputs, mean squared error is often used instead:

$$\mathcal{L} = \frac{1}{T} \sum_{t=1}^{T} \|\mathbf{y}_t - \hat{\mathbf{y}}_t\|^2. \tag{4}$$

With a one-hot target, (3) is exactly $-\sum_t \log p_\theta(x_t \mid x_{<t})$, so minimizing cross-entropy under teacher forcing is the same thing as maximizing (2).

Remark. Teacher forcing creates a mismatch between training and generation, often called *exposure bias*. At training time the model always sees correct prefixes; at generation time it must condition on its own earlier samples, and one early mistake can compound. Topic 3 returns to this when discussing decoding strategies.

1.4 Choosing an ordering

The chain rule holds for *any* ordering of the dimensions, but the model only ever sees one ordering, so the choice matters.

- **Text.** Left-to-right is the obvious choice and is what every modern language model uses. The reverse ordering is also a valid model and was historically used in some translation systems.
- **Images.** PixelCNN orders pixels in raster-scan order (top-left to bottom-right, row by row). This is somewhat arbitrary; pixels have no natural one-dimensional order, and the choice introduces directional artifacts (Topic 4).
- **Audio.** WaveNet orders samples in time, which is natural.
- **Code.** Tokenize left-to-right, the same as text.

Once chosen, the ordering is part of the model. Two PixelCNNs trained with different orderings will produce subtly different sample distributions even on the same data.

1.5 Exact likelihood: a real benefit

Compare the three families on whether you can ask "how likely is this observation under my model?"

- **VAE:** yes, but only a lower bound (the ELBO).
- **GAN:** no; the model has no explicit likelihood.
- **Autoregressive:** yes, exactly, via the product of conditionals.

Exact likelihood is useful for compression (the optimal code length for an observation is $-\log_2 p(x)$ bits), for outlier detection (low-likelihood observations are candidates), for model comparison (perplexity, which is exponentiated negative log-likelihood per token; see Topic 3), and for the theoretical guarantees that come with proper probabilistic modeling. It is the single most important practical difference between autoregressive and adversarial models.

1.6 Payoff

An autoregressive model is a factorization plus a neural network for each factor. Everything that follows this week is about the two engineering questions the factorization leaves open: how to build one network that can consume a prefix of arbitrary length (Topic 2), and how to turn a trained model into a useful generator (Topic 3).

Readings for Topic 1

- D. Foster, *Generative Deep Learning*, 2nd ed., O'Reilly, 2023, Chapter 5 (Autoregressive Models).
- A. van den Oord et al., "Pixel Recurrent Neural Networks," ICML 2016, arXiv:1601.06759 (Sections 1 and 2).
- A. van den Oord et al., "WaveNet: A Generative Model for Raw Audio," 2016, arXiv:1609.03499.
- I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning*, MIT Press, 2016, Chapter 10.

2 Topic 2: Recurrent Models and LSTM Intuition

2.1 Plain-English summary

Topic 1 said the autoregressive trick is to parameterize each per-position conditional $p_\theta(x_i \mid x_{<i})$ with a neural network. This topic is about *how* you build a network that takes a variable-length sequence as input. The original answer is the recurrent neural network (RNN): a small block of weights that you apply over and over, threading a hidden state through time. Vanilla RNNs train poorly on long sequences because of the vanishing-gradient problem. The LSTM cell, with its three gates and a separate cell state, is the fix that made long-sequence training tractable and dominated the field for two decades before transformers took over (Week 6).

The rest of this topic works through the RNN, the gated RNN (GRU), the LSTM, and the stacked LSTM with equations, a worked toy example for each, and the backpropagation-through-time intuition that explains why gates help.

2.2 Recurrent Neural Network (RNN)

An RNN processes sequential data by updating its hidden state using the current input and the previous hidden state. The same weight matrices are reused at every time step.

Equations

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h), \tag{5}$$
$$y_t = W_{hy}h_t + b_y. \tag{6}$$

Here:

- x_t: the input at time step t.
- h_t: the hidden state (memory) at time step t.
- y_t: the output at time step t.
- W_{xh}, W_{hh}, W_{hy}: weight matrices (shared across all time steps).
- b_h, b_y: bias terms.

For autoregressive generation, y_t is turned into the parameters of $p_\theta(x_{t+1} \mid x_{\leq t})$: a softmax over the vocabulary for categorical data, or a Gaussian mean and variance (or a mixture) for continuous data.

Training Process

Training involves:

i) Forward pass: Compute hidden states $\mathbf{h}_t$ and predictions $\mathbf{y}_t$ sequentially.

ii) Compute loss over all time steps.

iii) Backpropagation Through Time (BPTT): Unroll the RNN for T steps and propagate gradients backward through all steps.

iv) Update weights using gradient descent (or variants like Adam).

Step-by-Step Example

Suppose:

- Inputs: $x_1 = 1.0$, $x_2 = 2.0$
- Parameters: $W_{xh} = 0.5$, $W_{hh} = 0.1$, $W_{hy} = 1.0$
- Initial state: $h_0 = 0$

Step 1 (time $t = 1$):

$$h_1 = \tanh(0.5 \cdot 1.0 + 0.1 \cdot 0) = \tanh(0.5) \approx 0.462,$$
$$y_1 = 1.0 \cdot 0.462 = 0.462.$$

Step 2 (time $t = 2$):

$$h_2 = \tanh(0.5 \cdot 2.0 + 0.1 \cdot 0.462) = \tanh(1.0462) \approx 0.779,$$
$$y_2 = 0.779.$$

Comment: The hidden state h_t evolves over time, but for long sequences, gradients may vanish or explode during training.

Backpropagation Through Time (BPTT)

Given a loss $\mathcal{L} = \sum_t \ell(y_t, \hat{y}_t)$, BPTT unrolls the RNN across all time steps and applies the chain rule:

$$\frac{\partial \mathcal{L}}{\partial W_{xh}} = \sum_t \frac{\partial \mathcal{L}}{\partial h_t} \frac{\partial h_t}{\partial W_{xh}}, \tag{7}$$

$$\frac{\partial h_t}{\partial h_{t-1}} = (1 - h_t^2) W_{hh}. \tag{8}$$

Thus gradients propagate backward through time as repeated multiplications of Jacobians, which may shrink (vanishing gradient) or blow up (exploding gradient).

Why vanilla RNNs fail at long sequences

Over many time steps the gradient reaching an early step is a product of many Jacobians $(1 - h_t^2)W_{hh}$. Two failure modes follow:

- **Vanishing gradient:** if the eigenvalues of W_{hh} are smaller than 1 in magnitude, the gradient shrinks exponentially with sequence length. The network cannot learn long-range dependencies because the signal never reaches the early time steps.
- **Exploding gradient:** if the eigenvalues are larger than 1, the gradient grows exponentially and training becomes numerically unstable.

Exploding gradients can be controlled by gradient clipping. Vanishing gradients are the harder problem: the network has no useful signal to learn from. This is why vanilla RNNs struggle on dependencies more than roughly ten time steps apart, while real text or audio has dependencies thousands of steps long.

2.3 Gated RNN (GRU)

A Gated Recurrent Unit (GRU) modifies the RNN by introducing gates that control information flow, making training more stable than vanilla RNNs.

Equations

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z) \quad \text{(update gate)}, \tag{9}$$
$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r) \quad \text{(reset gate)}, \tag{10}$$
$$\tilde{h}_t = \tanh(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h) \quad \text{(candidate state)}, \tag{11}$$
$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t. \quad \text{(hidden state)} \tag{12}$$

Step-by-Step Example

Suppose input $x_1 = 1.0$, initial $h_0 = 0$, and weights $W_z = W_r = W_h = 1.0$, $U_z = U_r = U_h = 0.5$.

$$z_1 = \sigma(1.0) = 0.731,$$
$$r_1 = \sigma(1.0) = 0.731,$$
$$\tilde{h}_1 = \tanh(1.0 + 0.5 \cdot (0.731 \cdot 0)) = \tanh(1.0) = 0.762,$$
$$h_1 = (1 - 0.731) \cdot 0 + 0.731 \cdot 0.762 = 0.557.$$

Comment: The update gate z_t balances between keeping the past memory and updating with new information.

Backpropagation Intuition

The gradient with respect to h_{t-1} involves terms from both $(1 - z_t)$ and r_t, providing controlled gradient flow. Compared with RNNs, GRUs mitigate vanishing gradients by allowing direct paths for memory through the update gate.

2.4 Long Short-Term Memory (LSTM)

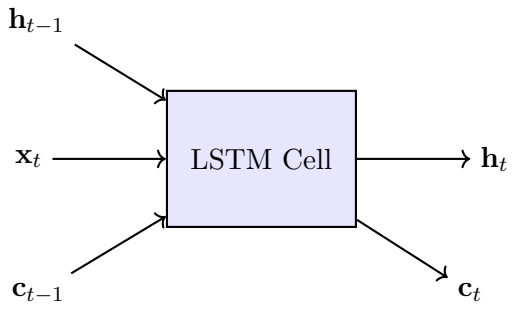
An LSTM improves on the RNN by introducing a **cell state** c_t and **gates** to regulate information flow. This structure allows gradients to propagate more stably.

Equations

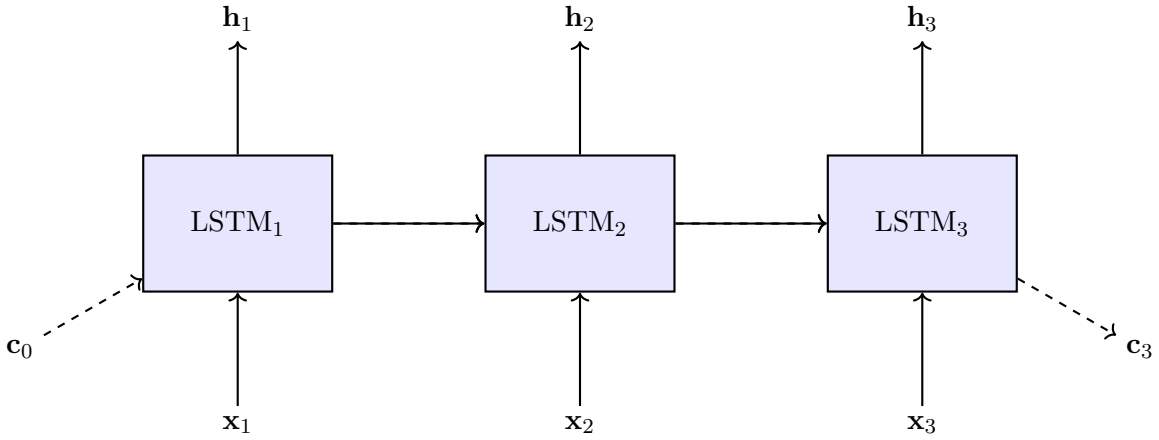
At each step, with input x_t, previous hidden state h_{t-1}, and previous cell state c_{t-1}:

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f) \quad \text{(forget gate)}, \tag{13}$$
$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i) \quad \text{(input gate)}, \tag{14}$$
$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c) \quad \text{(candidate state)}, \tag{15}$$
$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad \text{(update cell)}, \tag{16}$$
$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o) \quad \text{(output gate)}, \tag{17}$$
$$h_t = o_t \odot \tanh(c_t) \quad \text{(hidden state)}. \tag{18}$$

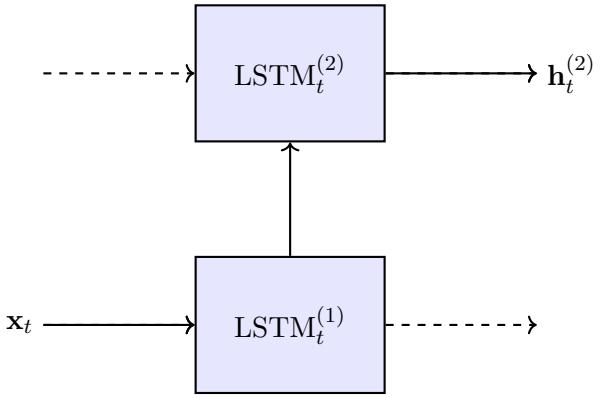
Many references (including Foster and the Olah blog post) write each gate with a single matrix acting on the concatenated vector $[\mathbf{h}_{t-1}, \mathbf{x}_t]$, for example $f_t = \sigma(W_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f)$. The two notations are equivalent: the concatenated W_f is just $[U_f\ W_f]$ stacked side by side. Figure 1 shows the cell, its unrolling over time, and a two-layer stack.



(a) Single LSTM cell (one time step)



(b) LSTM unrolled over time (shared parameters)



(c) Stacked LSTM (2 layers)

Figure 1: LSTM architectures. Solid arrows carry inputs and hidden states; dashed arrows carry the cell state $\mathbf{c}_t$. (a) One cell at one time step. (b) The same cell unrolled over three time steps; all three copies share one parameter set. (c) Two stacked layers at a single time step; the hidden state of layer 1 is the input to layer 2.

Training Process

Similar to RNNs, but gradients are better preserved:

i) Forward pass through LSTM cells, updating $\mathbf{h}_t$ and $\mathbf{c}_t$.

ii) Compute sequence loss (cross-entropy or MSE, as in Equations (3) and (4)).

iii) Backpropagation Through Time with LSTM cell derivatives (mitigating vanishing gradient problem).

iv) Parameter update using optimizers.

Step-by-Step Example

Suppose:

- Input: $x_1 = 1.0$
- Parameters: $W_f = W_i = W_c = W_o = 1.0$, $U_f = U_i = U_c = U_o = 0.5$
- Initial states: $h_0 = 0$, $c_0 = 0$

Step 1 (compute gates):

$$\begin{aligned} f_1 &= \sigma(1.0) = 0.731, \\ i_1 &= \sigma(1.0) = 0.731, \\ \tilde{c}_1 &= \tanh(1.0) = 0.762. \end{aligned}$$

Step 2 (update cell state):

$$c_1 = 0.731 \cdot 0 + 0.731 \cdot 0.762 = 0.557.$$

Step 3 (hidden state):

$$\begin{aligned} o_1 &= \sigma(1.0) = 0.731, \\ h_1 &= 0.731 \cdot \tanh(0.557) = 0.369. \end{aligned}$$

Comment: The cell state c_1 accumulates information additively, preserving memory over longer sequences.

Backpropagation Intuition

From $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$, we have:

$$\frac{\partial c_t}{\partial c_{t-1}} = f_t. \tag{19}$$

If $f_t \approx 1$, gradients flow nearly unchanged across time, alleviating vanishing gradients. Contrast this with the RNN Jacobian $(1 - h_t^2)W_{hh}$, which is squashed by the tanh derivative and rescaled by W_{hh} at every step. The cell state is a highway along which information can travel long distances without being squashed by repeated tanh operations; each gate is a learned switch that decides whether information enters, stays on, or leaves that highway.

2.5 Stacked LSTM

A stacked LSTM consists of multiple layers of LSTMs. The output of one layer becomes the input to the next, allowing the network to learn hierarchical features.

Equations

For layer ℓ ($1 \leq \ell \leq L$):

$$(h_t^{(\ell)}, c_t^{(\ell)}) = \text{LSTM}^{(\ell)}(h_t^{(\ell-1)}, h_{t-1}^{(\ell)}, c_{t-1}^{(\ell)}), \tag{20}$$

with $h_t^{(0)} = x_t$. Written out for L layers:

$$\mathbf{h}_t^{(1)}, \mathbf{c}_t^{(1)} = \text{LSTM}^{(1)}(\mathbf{x}_t, \mathbf{h}_{t-1}^{(1)}, \mathbf{c}_{t-1}^{(1)}), \qquad \mathbf{h}_t^{(2)}, \mathbf{c}_t^{(2)} = \text{LSTM}^{(2)}(\mathbf{h}_t^{(1)}, \mathbf{h}_{t-1}^{(2)}, \mathbf{c}_{t-1}^{(2)}), \qquad \dots$$

$$\mathbf{h}_t^{(L)}, \mathbf{c}_t^{(L)} = \text{LSTM}^{(L)}(\mathbf{h}_t^{(L-1)}, \mathbf{h}_{t-1}^{(L)}, \mathbf{c}_{t-1}^{(L)}).$$

Final output is produced from the top layer $\mathbf{h}_t^{(L)}$.

Training Process

i) Forward pass through all L stacked layers.

ii) Compute sequence loss from top-layer outputs.

iii) Apply BPTT across both time steps and layers.

iv) Update parameters of all LSTM layers jointly.

The loss remains the same (cross-entropy or MSE), applied at the top layer's output.

Step-by-Step Example

Suppose the first layer outputs $h_1^{(1)} = 0.369$. This becomes the input to the second LSTM layer (with the same weights as the layer-1 toy example).

Step 1 (compute gates in layer 2):

$$\begin{aligned} f_1^{(2)} &= \sigma(0.369) = 0.591, \\ i_1^{(2)} &= 0.591, \\ \tilde{c}_1^{(2)} &= \tanh(0.369) = 0.353. \end{aligned}$$

Step 2 (update cell in layer 2):

$$c_1^{(2)} = 0.591 \cdot 0 + 0.591 \cdot 0.353 = 0.209.$$

Step 3 (compute hidden state in layer 2):

$$\begin{aligned} o_1^{(2)} &= 0.591, \\ h_1^{(2)} &= 0.591 \cdot \tanh(0.209) = 0.122. \end{aligned}$$

Comment: Layer 1 extracts features directly from the raw input. Layer 2 processes these features further, building hierarchical representations.

2.6 Parameter Sharing and Backpropagation in LSTMs

Parameter Sharing Across Time

An LSTM cell has learnable parameters for its gates:

$$\theta = \{W_f, W_i, W_c, W_o, U_f, U_i, U_c, U_o, b_f, b_i, b_c, b_o\}.$$

When processing a sequence $\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_T$, the LSTM is *unrolled* into T cells (one per time step). Importantly, all T cells share the same parameters θ. This parameter sharing:

- keeps the number of parameters independent of sequence length,
- allows the network to generalize patterns across different positions in the sequence.

Thus, while we draw multiple "copies" of the LSTM cell in diagrams (Figure 1b), mathematically there is only one set of weights applied repeatedly.

Parameter Sharing Across Layers

In a *stacked LSTM*, each layer has its own parameter set. For L layers, we have

$$\theta^{(1)}, \theta^{(2)}, \ldots, \theta^{(L)},$$

where parameters are shared across time within a layer but differ across layers.

Backpropagation Through Time (BPTT)

During training, the loss is typically defined as the sum over time steps:

$$\mathcal{L} = \sum_{t=1}^{T} \mathcal{L}_t.$$

When computing gradients with respect to a shared parameter $W \in \theta$, we accumulate contributions from all time steps:

$$\frac{\partial \mathcal{L}}{\partial W} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}_t}{\partial W}.$$

This ensures that weight updates reflect the influence of the entire sequence.
Remarks.

- Across **time steps**: one parameter set is shared (no parameter explosion).
- Across **layers**: each layer has its own parameters.
- Backpropagation computes gradients through the unrolled network and accumulates them for the shared parameters.

2.7 Comparison and Discussion

- **RNN**: Simplest recurrent model. Struggles with long-term dependencies due to vanishing/exploding gradients.
- **Gated RNN (GRU)**: Introduces reset and update gates. More efficient than LSTM (fewer parameters) but less expressive in some tasks. Often performs competitively.
- **LSTM**: Introduces cell state with input, forget, and output gates. Best at capturing long-range dependencies. More parameters than GRU.
- **Stacked LSTM**: Adds depth to capture hierarchical and abstract sequence patterns.

When to use:

- RNNs: For short sequences or when simplicity matters.
- GRUs: When training data is limited and efficiency is important.
- LSTMs: When modeling long-term dependencies is crucial.
- Stacked LSTMs: When tasks demand high capacity and abstraction.

One more variant you will hear about. A *bidirectional* RNN or LSTM runs one recurrence forward and one backward over the sequence and concatenates the hidden states. It is useful when the whole sequence is available at training time (sequence labeling, the encoder of a translation system). It is *not* usable for autoregressive generation, where only the past is available when each token is produced.

2.8 Payoff (Key Takeaways)

- **RNN**: Direct recurrence. Simple, but struggles with long-term dependencies.
- **Gated RNN (GRU)**: Uses gates to mitigate vanishing gradients while being simpler than LSTMs.
- **LSTM**: Adds gates and a cell state to preserve information across long time spans.
- **Stacked LSTM**: Multiple layers enable deeper, more abstract sequence modeling.

You do not need to memorize the gate equations. The intuition you do need: each gate is a learned switch that decides whether information flows through, and the cell state is a highway that lets information (and gradients) travel long distances. Week 6 replaces the recurrence with self-attention; the autoregressive factorization of Topic 1 stays exactly the same.

Readings for Topic 2

- D. Foster, *Generative Deep Learning*, 2nd ed., Chapter 5, sections on RNN and LSTM architectures (companion notebook `05_autoregressive/01_lstm`).
- S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation* 9(8), 1997.
- C. Olah, "Understanding LSTM Networks," blog post, 2015.
- K. Cho et al., "Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation," EMNLP 2014, arXiv:1406.1078 (the GRU paper).

3 Topic 3: Sampling, Conditioning, and Sequence Quality

3.1 Plain-English summary

Training an autoregressive model is the friendliest objective in deep learning: maximize the sum of next-token log-probabilities. Sampling from a trained model is where the engineering lives. The same model can produce dull, repetitive text (greedy decoding) or wildly incoherent text (high-temperature sampling) depending on which decoding strategy you pick. This topic walks through the standard decoding strategies, the conditioning patterns that let you steer the generator, the quality metrics you can compute, and the asymptotic cost of autoregressive sampling that nobody can engineer around.

3.2 Decoding strategies: how to pick the next token

At each step i of generation the model returns a distribution $p_\theta(x_i \mid x_{<i})$ over possible next tokens (or pixels, or audio samples). You have to pick one. Write z_k for the logit of token k, so that $p_\theta(x_i = k \mid x_{<i}) = \text{softmax}(z)_k = e^{z_k} / \sum_j e^{z_j}$.

Greedy decoding. Always pick the highest-probability token,

$$x_i = \arg\max_k \; p_\theta(x_i = k \mid x_{<i}). \tag{21}$$

Deterministic given the prefix and fast, but it produces repetitive, low-diversity output because the highest-probability path is often dominated by common short tokens. Almost never the right choice for open-ended generation.

Temperature sampling. Sharpen or flatten the distribution before sampling:

$$p_T(x_i = k \mid x_{<i}) \;\propto\; p_\theta(x_i = k \mid x_{<i})^{1/T} \quad \Longleftrightarrow \quad p_T(x_i = k \mid x_{<i}) = \frac{e^{z_k/T}}{\sum_j e^{z_j/T}}, \qquad T > 0. \tag{22}$$

In plain language: $T = 1$ leaves the distribution unchanged; $T < 1$ sharpens it (closer to greedy; more deterministic, less diverse); $T > 1$ flattens it (more random, more diverse, more incoherent). As $T \to 0$ temperature sampling becomes greedy decoding; as $T \to \infty$ it becomes uniform over the vocabulary. For text, T between 0.7 and 1.0 is the typical sweet spot; creative tasks push higher and factual completion lower. It is one of the few settings you tune by eye, and Foster's `01_lstm` notebook implements it in a few lines.

Top-k sampling. Keep only the k highest-probability tokens, renormalize, and sample from those. This cuts off the long tail of low-probability tokens that would otherwise occasionally produce nonsense. Typical k for text: 20 to 50.

Top-p (nucleus) sampling. Holtzman et al. (2020) keep the smallest set V_p of top tokens whose cumulative probability exceeds p,

$$V_p = \text{smallest } V \text{ (by descending probability) such that } \sum_{k \in V} p_\theta(x_i = k \mid x_{<i}) \geq p, \tag{23}$$

then renormalize and sample within V_p. The nucleus adapts to the model's confidence: when the model is sure, only a few tokens clear the threshold; when it is uncertain, the set expands. Typical p: 0.9 to 0.95. Generally preferred over top-k for natural-language generation.

Beam search. Maintain B candidate prefixes at each step, expand each by all possible next tokens, and keep the B extended candidates with the highest total log-probability $\sum_{j \le i} \log p_\theta(x_j \mid x_{<j})$. Return the best complete sequence. Beam search is used where one best output matters more than diversity (machine translation, summarization, structured prediction). It is not used for open-ended generation because the approximately most-likely sequence is highly repetitive.

3.3 Conditioning: steering what the model produces

The vanilla autoregressive model has no knobs at inference: it samples $x_1 \sim p(x_1)$, then $x_2 \sim p(x_2 \mid x_1)$, and so on. To steer it, you add conditioning information y that the model was trained to respect, so that every factor becomes $p_\theta(x_i \mid x_{<i}, y)$:

- **Prompt-based.** Prepend a token sequence y to the input. The model continues from there. This is how language models work: there is no special "prompt" mechanism; the prompt is simply the start of the sequence.
- **Class-conditional.** Train the model to take a class label as an additional input (concatenated to each time step's input, or used to initialize the hidden state). Foster's class-conditional PixelCNN is the example.
- **Encoder context (seq2seq).** An encoder network compresses an input sequence into a context vector; the autoregressive decoder takes that as its initial state. This is the Sutskever et al. (2014) machine-translation setup, and the structure of every later encoder-decoder model.

3.4 Quality metrics for autoregressive models

- **Perplexity.** Exponentiated average negative log-likelihood per token,

$$\mathrm{PPL} = \exp\left(-\frac{1}{N} \sum_{i=1}^{N} \log p_\theta(x_i \mid x_{<i}) \right). \tag{24}$$

  Lower is better. Native to autoregressive models because they have exact likelihood, and the standard metric for language models on held-out data. Use with caution: perplexity measures how well the model predicts your test set, not how good its samples look to a human. A model can have low perplexity and still generate incoherent text, because predicting the next token well (given a good prefix) and producing a good sequence (given your own imperfect prefix) are different tasks; this is the exposure-bias point from Topic 1.

- **BLEU, ROUGE, METEOR.** Reference-based metrics for translation and summarization. Useful when a ground-truth output exists; useless for open-ended generation.
- **FID (image autoregressive).** The same metric as for GANs in Week 3. Treat the autoregressive model as a sample source and compute FID against real images.
- **Human evaluation.** Still the gold standard for open-ended generation. Quantitative metrics correlate imperfectly with human judgment.

3.5 The unavoidable cost: $O(N)$ sampling

To sample a length-N sequence you must run the model N times, once per token, and each step depends on all previous tokens, so the steps cannot be parallelized. This is intrinsic to the factorization (1), not a software issue. Training does not have this problem: under teacher forcing all N positions are scored in one pass.

For a 100-token text response on a modest LSTM the cost is invisible. For a 1024×1024 image on a PixelCNN (over one million pixels) it is brutal: hours of inference per image. The same problem affects WaveNet on raw audio, and it is the main reason diffusion models (Week 5), which denoise all positions in parallel, have largely displaced autoregressive image models for high-resolution work.

3.6 Payoff

A trained autoregressive model is only half of a generator; the decoding strategy is the other half. Temperature, top-k, nucleus, and beam search all read the same $p_\theta(x_i \mid x_{<i})$ and differ only in how they turn it into a choice. The Week 4 mini-lab asks you to run one Foster notebook with at least one decoding variation so that these knobs show up in your own samples.

Readings for Topic 3

- D. Foster, *Generative Deep Learning*, 2nd ed., Chapter 5, sections on generation and sampling.
- A. Holtzman et al., "The Curious Case of Neural Text Degeneration," ICLR 2020, arXiv:1904.09751 (Sections 1 to 4).
- I. Sutskever, O. Vinyals, Q. V. Le, "Sequence to Sequence Learning with Neural Networks," NeurIPS 2014, arXiv:1409.3215.
- D. Bahdanau, K. Cho, Y. Bengio, "Neural Machine Translation by Jointly Learning to Align and Translate," ICLR 2015, arXiv:1409.0473.

4 Topic 4: Strengths and Limitations of Autoregressive Generation

4.1 Plain-English summary

Autoregressive is the family behind the most widely deployed generative-AI systems today: every large language model, most code-completion systems, WaveNet-style audio synthesis, PixelCNN-style image generation, and protein-sequence design. It earned that position by being the easiest family to train with the most defensible probabilistic story. It also has hard limits: sampling is sequential and slow, the choice of ordering biases the model, and very long-range structure remained difficult until transformers solved a large part of it (Week 6). This topic puts autoregressive models side by side with VAEs (Week 2) and GANs (Week 3) so you can pick the right family for your project.

4.2 Where autoregressive shines

- **Exact likelihood.** No ELBO gap, no adversarial fixed point. The objective $\sum_i \log p_\theta(x_i \mid x_{<i})$ is well-conditioned, well-understood, and parallelizes across positions during training (even though sampling is sequential).
- **Broad applicability.** Any data with a natural ordering works: text, code, audio, DNA, time series, and images via raster scan. The factorization $p(x) = \prod_i p(x_i \mid x_{<i})$ is universal in a way the latent-variable and adversarial setups are not.
- **Strong tooling.** Hugging Face Transformers, PyTorch LSTMs, JAX models; every major framework has first-class autoregressive support and pre-trained checkpoints.
- **Scales gracefully.** Bigger models, more data, and longer context windows keep improving performance (in a way GANs do not). The scaling-laws literature (Kaplan et al., 2020) is almost entirely about autoregressive models.
- **Conditioning is easy.** Prepend the conditioning tokens, or start from a start-of-sequence token. The same mechanism handles everything.

4.3 Where autoregressive struggles

- **Sampling cost is $O(N)$ and not parallelizable across positions.** Every sample needs N forward passes. For long sequences (high-resolution images, long-form audio) this is the bottleneck.
- **The ordering choice biases the model.** A raster-scan PixelCNN sees the upper-left corner first and uses it to predict the rest; samples inherit a directional bias. There is no symmetric way to run autoregression on intrinsically unordered data.
- **No bidirectional conditioning.** Modeling $p(x_i \mid x_{>i})$ (useful for infilling) needs a separate model or a non-autoregressive setup. BERT-style masked language modeling exists but is not a proper probabilistic model of the joint.
- **Diversity and mode coverage depend on the sampler.** The same model gives excellent text at $T = 0.8$, nonsense at $T = 1.5$, and boring repetition at $T = 0$. Without a temperature knob you get one operating point.

- **Long-range dependencies are hard for vanilla RNN/LSTM.** Even the LSTM struggles past a few hundred time steps. Transformers are the architectural fix.

4.4 VAE versus GAN versus autoregressive

Table 1 extends the two-column comparison from Week 3 with a third column.

Dimension	**VAE**	**GAN**	**Autoregressive**
Training objective	ELBO (lower bound on log-likelihood)	Minimax saddle point; no likelihood	Exact log-likelihood (sum of per-token terms)
Latent variable	Yes, with encoder $q_\phi(z \mid x)$	Implicit (noise z into generator)	None
Sample sharpness	Blurry (Gaussian decoder)	Sharp when training succeeds	Sharp; sequence quality varies with sampling strategy
Mode coverage	Broad (KL regularizer)	Can drop modes	Broad in principle; sampler-dependent
Training stability	Routine	Hard; many failure modes	Routine
Controllability	Latent arithmetic on z	Conditional GAN, StyleGAN	Prompt-based conditioning
Likelihood evaluation	Lower bound only	None	Exact
Sampling cost	One forward pass	One forward pass	$O(N)$ sequential forward passes
Typical use cases	Representation learning, anomaly detection	High-quality image synthesis	Language, code, audio, DNA, anything with order

Table 1: The three generative-model families covered so far, compared on the same axes.

4.5 Forward reference: transformers and diffusion

Two developments since the LSTM era changed the field without changing the factorization of Topic 1.

- **Transformers (Week 6)** replaced recurrence with self-attention. Each factor $p_\theta(x_i \mid x_{<i})$ is still there; the function computing it becomes self-attention over the prefix instead of an unrolled recurrence. This made it possible to train much deeper and wider autoregressive models on much longer sequences. Every modern large language model is an autoregressive transformer.
- **Diffusion (Week 5)** sidesteps the $O(N)$ sampling cost by training a model that denoises all positions in parallel over T steps (typically 50 to 1000). Diffusion has largely displaced PixelCNN-style models for high-resolution images while leaving text and audio mostly to autoregressive transformers.

4.6 Payoff

You now have a working understanding of the third major family. When choosing a model for a project, the decisive questions are usually the ones in Table 1: Do you need a likelihood? Do you need a latent space? Is the data naturally ordered? Is $O(N)$ sampling acceptable? Autoregressive models answer "yes, no, yes, it depends," and that profile explains both why they run the largest deployed systems and why diffusion took over high-resolution images.

Readings for Topic 4

- D. Foster, *Generative Deep Learning*, 2nd ed., Chapter 5, concluding sections.
- J. Kaplan et al., "Scaling Laws for Neural Language Models," 2020, arXiv:2001.08361.
- A. Vaswani et al., "Attention Is All You Need," NeurIPS 2017, arXiv:1706.03762.
- T. Brown et al., "Language Models are Few-Shot Learners," NeurIPS 2020, arXiv:2005.14165.